

Calculus, Autodiff & Optimization: Worked Solutions

Problem 1. Row and column chain rules

Problem. For $f : \mathbb{R}^3 \rightarrow \mathbb{R}^2$ and scalar $L : \mathbb{R}^2 \rightarrow \mathbb{R}$, write the dimensions of J_f , $D_f L$, $D_x(L \circ f)$, and $\nabla_x(L \circ f)$. Derive both the row-derivative and column-gradient chain rules.

Solution. Let $y = f(x)$, and write $D_y L|_{y=f(x)}$ for the derivative denoted by $D_f L$ in the question. Assume differentiability at the relevant points. Numerator-layout Jacobians place output indices first, so

$$J_f(x) = \left[\frac{\partial f_i}{\partial x_j} \right]_{i,j} \in \mathbb{R}^{2 \times 3}, \quad D_y L(f(x)) \in \mathbb{R}^{1 \times 2}.$$

Since $dy = J_f(x)dx$ and $dL = D_y L dy$,

$$d(L \circ f) = D_y L(f(x))J_f(x)dx.$$

The row derivative and its Euclidean column gradient are therefore

$$D_x(L \circ f)(x) = D_y L(f(x))J_f(x) \in \mathbb{R}^{1 \times 3},$$

$$\nabla_x(L \circ f)(x) = J_f(x)^T \nabla_y L(f(x)) \in \mathbb{R}^{3 \times 1}.$$

The second rule is the transpose of the first. In either convention, the derivative sums contributions through both intermediate coordinates: the j th input partial is $\sum_{i=1}^2 (\partial L / \partial y_i)(\partial f_i / \partial x_j)$.

Problem 2. A quadratic with a nonsymmetric matrix

Problem. Differentiate $x^T A x$ without assuming symmetry. Use a differential to obtain the gradient and Hessian. Explain why the skew-symmetric part of A cannot affect the scalar value.

Solution. Let $x \in \mathbb{R}^d$ and keep $A \in \mathbb{R}^{d \times d}$ fixed. The product rule gives

$$d(x^T A x) = (dx)^T A x + x^T A dx = x^T (A^T + A) dx.$$

Comparing with $dF = (\nabla F)^T dx$ yields

$$\nabla_x(x^T A x) = (A + A^T)x, \quad \nabla_x^2(x^T A x) = A + A^T.$$

The factor two is valid only after replacing A by its symmetric part or assuming $A = A^T$.

Decompose $A = S + K$, where $S = (A + A^T)/2$ is symmetric and $K = (A - A^T)/2$ satisfies $K^T = -K$. The scalar $s = x^T K x$ equals its transpose, but

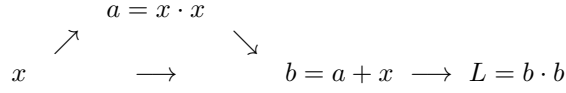
$$s = s^T = x^T K^T x = -x^T K x = -s.$$

Thus $s = 0$ over the real numbers, and $x^T A x = x^T S x$. The gradient is accordingly $2Sx$, and the Hessian is $2S$. The skew-symmetric component affects neither the value nor any of its derivatives with respect to x .

Problem 3. Accumulating paths in reverse mode

Problem. For $L(x) = (x^2 + x)^2$, draw the scalar computational graph and accumulate all paths to x . Evaluate the derivative at $x = 2$. Explain which term is lost if a backward implementation overwrites one contribution.

Solution. Use the nodes $a = x \cdot x$, $b = a + x$, and $L = b \cdot b$. The following dependency diagram labels repeated input slots explicitly:



For clarity, the edges are $x \rightarrow a$ twice, $a \rightarrow b$, $x \rightarrow b$, and $b \rightarrow L$ twice. Repeated input slots contribute separately even when they refer to the same node.

Write $\bar{v} = \partial L / \partial v$ and seed $\bar{L} = 1$. The multiplication $L = b \cdot b$ gives $\bar{b} = b + b = 2b$. The addition $b = a + x$ gives $\bar{a} = 2b$ and a direct contribution $2b$ to $\bar{x}$. Finally, the two input slots of $a = x \cdot x$ each contribute $x\bar{a} = 2bx$. Accumulating all contributions gives

$$\bar{x} = 2b + 2bx + 2bx = 2(x^2 + x)(2x + 1).$$

At $x = 2$, the forward values are $a = 4$, $b = 6$, and $L = 36$. The direct path contributes 12 and the two multiplication paths contribute 24 each, so $L'(2) = 12 + 24 + 24 = 60$.

Overwriting the direct contribution with the combined contribution through a would return 48, omitting the derivative of the $+x$ term. Overwriting in the other order would return 12, omitting the derivative through x^2 . A separate aliasing error—keeping only one input-slot contribution in $x \cdot x$ —would return 36 if the direct contribution were otherwise retained. The correct operation at shared nodes is addition, not assignment.

Problem 4. Loss reductions and duplicated batches

Problem. Let $X \in \mathbb{R}^{B \times d}$ and $Y \in \mathbb{R}^{B \times k}$ be row batches. With $W_1 \in \mathbb{R}^{h \times d}$, $b_1 \in \mathbb{R}^h$, $W_2 \in \mathbb{R}^{k \times h}$, and $b_2 \in \mathbb{R}^k$, consider

$$A = XW_1^T + \mathbf{1}_B b_1^T, \quad H = \phi(A), \quad Z = HW_2^T + \mathbf{1}_B b_2^T,$$

where ϕ acts coordinatewise and is differentiable at the evaluated entries. Replace $L = \|Z - Y\|_F^2 / (2B)$ by $\tilde{L} = \|Z - Y\|_F^2 / (2Bk)$. State every changed gradient factor. Then duplicate every row of X, Y and compare parameter and input gradients.

Solution. For the original objective, the output gradient and the hidden preactivation gradient are

$$\Delta_2 = \frac{Z - Y}{B} \in \mathbb{R}^{B \times k}, \quad \Delta_1 = (\Delta_2 W_2) \odot \phi'(A) \in \mathbb{R}^{B \times h}.$$

The differential of each affine operation yields

$$\begin{aligned}
 \nabla_{W_2} L &= \Delta_2^T H, & \nabla_{b_2} L &= \Delta_2^T \mathbf{1}_B, & \nabla_H L &= \Delta_2 W_2, \\
 \nabla_{W_1} L &= \Delta_1^T X, & \nabla_{b_1} L &= \Delta_1^T \mathbf{1}_B, & \nabla_X L &= \Delta_1 W_1.
 \end{aligned}$$

Matrix gradients have the shapes of their parameters; bias gradients are columns. The target gradient, if targets are differentiated, is $\nabla_Y L = (Y - Z)/B$.

Since $\tilde{L} = L/k$, every derivative with respect to the same argument is divided by k :

$$\tilde{\Delta}_2 = \frac{Z - Y}{Bk} = \frac{\Delta_2}{k}, \quad \tilde{\Delta}_1 = \frac{\Delta_1}{k}.$$

The six displayed gradients, $\nabla_Y L$, and the preactivation/output gradients all acquire exactly this one factor $1/k$. It enters at the loss and propagates through the chain rule. Dividing again by k at every layer would incorrectly introduce higher powers of $1/k$.

Now form $X' = [X^T, X^T]^T$ and $Y' = [Y^T, Y^T]^T$, with batch size $B' = 2B$. For either reduction, the squared-residual sum doubles and the denominator doubles, leaving the loss unchanged. Each output-gradient row is half its previous value. For $\tilde{L}$, for example,

$$\tilde{\Delta}'_2 = \frac{1}{2Bk} \begin{bmatrix} Z - Y \\ Z - Y \end{bmatrix} = \frac{1}{2} \begin{bmatrix} \tilde{\Delta}_2 \\ \tilde{\Delta}_2 \end{bmatrix}.$$

The same relation holds for the hidden gradients. Each parameter gradient sums over both copies, so its two half-sized contributions restore the original gradient:

$$\nabla_{W_j} \tilde{L}' = \nabla_{W_j} \tilde{L}, \quad \nabla_{b_j} \tilde{L}' = \nabla_{b_j} \tilde{L} \quad (j = 1, 2).$$

As independent coordinates, the two copied input rows each have half the original input gradient. Stacking these row gradients gives

$$\nabla_{X'} \tilde{L}' = \frac{1}{2} \begin{bmatrix} \nabla_X \tilde{L} \\ \nabla_X \tilde{L} \end{bmatrix}.$$

If duplication is itself a differentiable operation on the original X , its reverse rule sums the two copies, recovering $\nabla_X \tilde{L}$. Thus duplicated storage and differentiation with respect to the original shared input are distinct questions.

Problem 5. Exact gradient-descent dynamics

Problem. For $H = \text{diag}(1, 20)$ and $F(x) = x^T H x / 2$, determine the constant-step convergence interval. Starting from $(1, 1)^T$, compare steps $\eta = 0.05, 0.1, 0.12$. Which direction restricts stability and which slows convergence?

Solution. Here $\nabla F(x) = Hx$, so $x_{t+1} = (I - \eta H)x_t$. The two coordinate recurrences are independent:

$$x_{t,1} = (1 - \eta)^t x_{0,1}, \quad x_{t,2} = (1 - 20\eta)^t x_{0,2}.$$

Convergence to the minimizer 0 from every starting point requires both multipliers to have magnitude less than one. Intersecting $0 < \eta < 2$ with $0 < \eta < 2/20$ gives $0 < \eta < 0.1$.

For the specified initial point,

η	$x_{t,1}$	$x_{t,2}$	Behavior
0.05	0.95^t	$0 \ (t \geq 1)$	Converges; second coordinate vanishes in one step
0.10	0.90^t	$(-1)^t$	Second coordinate oscillates without decay
0.12	0.88^t	$(-1.4)^t$	Second coordinate alternates and grows

At the boundary step 0.1, $F(x_t) = \frac{1}{2}(0.9^{2t} + 20)$ tends to 10, not the minimum zero. At 0.12, the term $10(1.4)^{2t}$ makes the objective diverge. The curvature-20 direction restricts the largest stable step. For the stable step 0.05, the curvature-1 direction is the slow one, contracting by 0.95 per update. A vanishing initial component can hide instability in that direction; the interval above guarantees convergence for every initial point.

Problem 6. Momentum and optimizer state

Problem. Use the momentum convention

$$v_{t+1} = \beta v_t + g_t, \quad \theta_{t+1} = \theta_t - \eta v_{t+1}.$$

For $\beta = 0.8$, zero initial velocity, and constant scalar gradient two, compute the first three velocities. Explain why resetting the velocity while keeping the current parameter changes the next update.

Solution. With $v_0 = 0$ and $g_t = 2$,

$$v_1 = 0.8(0) + 2 = 2, \quad v_2 = 0.8(2) + 2 = 3.6, \quad v_3 = 0.8(3.6) + 2 = 4.88.$$

More generally, unrolling the recurrence gives

$$v_t = 2 \sum_{j=0}^{t-1} 0.8^j = \frac{2(1 - 0.8^t)}{1 - 0.8} = 10(1 - 0.8^t).$$

Continuing from (θ_t, v_t) gives the next parameter $\theta_t - \eta(0.8v_t + 2)$. Restarting from $(\theta_t, 0)$ instead gives $\theta_t - 2\eta$. The restarted parameter minus the continued parameter is $0.8\eta v_t$. For example, after the third update, continuing uses $v_4 = 0.8(4.88) + 2 = 5.904$, whereas resetting uses velocity two. The next parameters differ by 3.904η .

The velocity records past gradients, so a parameter value alone does not specify the state of this optimizer. With the alternative convention $v_{t+1} = \beta v_t + (1 - \beta)g_t$, all velocities in this constant-gradient example would be multiplied by $1 - \beta = 0.2$. The numerical values above correspond specifically to the stated unnormalized convention.